

今どきの画像アセット入稿

たった1枚の画像でアプリサイズが50MB増えた失敗から学ぶ最適化方法

上ちょ / uetyo (@psnzbss)

1 はじめに

以前、デザイナーから受け取ったベクター素材を SVG 形式で書き出し、Asset Catalog のオプションを "Preserve Vector Data: True", "Devices: Universal", "Scales: Single Scale" で格納したところ、アプリサイズが約 50MB 肥大化しました。追加したファイルは複雑なイラストだったものの、サイズは約 2MB で 50MB もの増加は見当もつきませんでした。本記事では、この失敗を基に "今どきの画像入稿方法" について調査・解説します^{*1}。

2 アプリに格納した画像ファイルが App Store で配信されるまでの流れ

Asset Catalog に格納された画像は、ビルド時に **actool** によって、ファイル形式や入稿オプション、アプリの Deployment Target に応じて処理され、Assets.car が生成されます。これらの処理には、画像の変換（ラスタライズ）、スケール別画像の生成（リサイジング）、ベクターデータの保持などがあり、アプリ配信時の最適化を容易にするために行われます。その後、App Store へ提出したアプリには、配信先のデバイスや OS バージョン、格納されたファイル形式に合わせて最適化（App Thinning）が適用され、表示に必要な最低限のファイルのみが含まれたアプリバイナリが生成されます（図 1）。

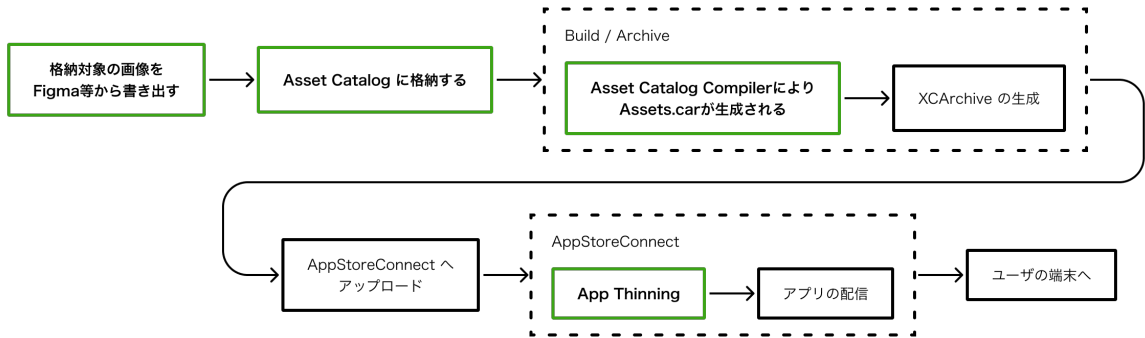


図 1 画像書き出しから配信までの流れ（概観）

3 ファイル形式・オプション指定の組み合わせ検証

本記事の執筆にあたり、ファイル形式・格納時のオプション指定がどのように影響するのか、実際にサンプルプロジェクトを作成して検証しました^{*2}。

3.1 検証に用いた画像とファイル形式

検証に際し、純粋なベクター画像（図 2）を作成しました。PNG・JPEG・SVG・PDF 形式への書き出しは Figma の標準機能を用いました。Figma から HEIC 形式への書き出しは、現時点でサポートされていないため、PNG 形式で書き出した後、**sips** コマンドで変換したものを利用しました。

```
sips -s format heic "target_image.png" --out "target_image.heic"
```

※ WebP 形式は Xcode（actool）でサポートされていないため対象外としています。



図 2 検証に用いた画像

3.2 Asset Catalog のオプション指定

各形式と下記の Asset Catalog オプションを機械的に組み合わせ、計 150 パターンを検証しました。

- Compression: Inherited / Lossless / Lossy-Basic / GPU Best Quality / GPU Smallest Size
- Resizing(Preserve Vector Data): True / False
- Scales: Single / Individual / Individual and Single

3.3 計測対象

各プロジェクトを iOS の Release としてアーカイブし、以下の 2 種類を計測しました。

1. Assets.car のサイズ: **Assets.car** のサイズを **stat** コマンドを用いて算出
2. アプリ配信時の推定サイズ^{*3}: **xcodebuild -exportArchive** に thinning オプション (**thin-for-all-variants**) を指定してビルドすると出力される App Thinning Size Report の variant 別サイズ (compressed / uncompressed) から算出

表 1 各形式と書き出し直後のファイルサイズ

ファイル形式	ファイルサイズ (MB)
JPEG (3x)	3.56
PNG (3x)	2.56
HEIC (3x)	0.57
PDF	0.92
SVG	0.38

^{*1} 画像そのものに対する技術的設定（インターレースやカラープロファイル等）は本記事では扱いません。
詳しくは Human Interface Guidelines (<https://developer.apple.com/design/human-interface-guidelines/images#Formats>) をご確認ください
^{*2} [Xcode] 26.6.0(17F113) [actool] 26.6(24765) [Supported Destinations and Minimum Deployments] iOS(iPadOS) / macOS / visionOS 26.0+
^{*3} App Store で配信される際のアプリサイズは、ローカルでの App Thinning 以上に高度に最適化が入っていると言われているため推定値です

4 検証結果

Assets.car に格納した画像が、ビルドや App Thinning でどのくらい影響を受けるのか確認しました^{*4}。

4.1 元ファイルと Assets.car のサイズは一致しない

Asset Catalog に格納した画像の合計サイズとビルドで生成される Assets.car には差があります。これは、ビルド時にファイル形式や Asset Catalog のオプション指定によって **actool** が再エンコードしたり追加生成したりすることがあるためです（表 2）。より詳細にサイズ差の原因を調べるため、**assetutil --info Assets.car** を実行し、Assets.car に格納された画像データ（レンディション）の形式や圧縮方式を確認しました。その結果、格納したファイルの形式や Compression によって内部の圧縮率やファイル数に違いがあることが判明しました。

表 2 格納したファイルとビルドで生成された Assets.car のサイズ（一部のみ掲載）

ファイル形式	Compression	Scales	元素材の合計（MB）	Assets.car サイズ（MB）	増加量（MB）
JPEG	Inherited	Individual	6.27	6.29	+0.02
PNG	GPU Best	Individual	4.68	6.67	+1.99
PNG	GPU Smallest	Individual	4.68	4.59	-0.09
PNG	Inherited	Individual	4.68	3.87	-0.81
PNG	Lossy-Basic	Individual	4.68	3.69	-0.99
HEIC	Inherited	Individual	1.07	5.84	+4.77
PDF	Inherited	Single	0.92	4.58	+3.66
SVG	Inherited	Single	0.38	2.97	+2.59

HEIC 形式は、自動生成された互換用ビットマップ（RGB555 + lzfse）と元ファイル（HEIC）の 2 つのレンディション^{*5}が保持されるため肥大化します。なお、今回の HEIC 素材と Xcode 26.6 の組み合わせでは、Compression を変更しても出力内容とサイズに差は観測できませんでした。

JPEG 形式は、Compression が Inherited や Lossless、Lossy-Basic の場合、元ファイルそのままの JPEG が格納されます。しかし、GPU Best / GPU Smallest を選択した場合は再処理されるためサイズが約 2 倍になります。

PNG 形式は少し特殊で、いずれの Compression 設定でも Apple 独自の形式に変換されます。Inherited や Lossless の場合、ARGB + deepmap2 が用いられることで今回の検証では約 18% 小さくなりました。Lossy-Basic の場合、RGB555 + lzfse を用いて色深度を 32bit → 16bit に削減することでサイズが小さくなります。なお、GPU Best / GPU Smallest を選択した場合、JPEG 同様にエンコードされたファイルが追加されるため、サイズは同じくらいか、大きく増加します。

PDF や SVG といったベクター画像は、アプリ表示時にそのまま利用されるわけではなく、ラスタライズされた 1x-3x の画像が用いられます。表 2 のように元ファイルのサイズが十分に小さい場合でも、ビルドプロセスを経由することで当初の 10 倍以上になる場合があります。また、どちらの形式でも元のベクターファイルは Assets.car に追加されますが、SVG 形式では、lzfse で圧縮されたものが配置されます（PDF は非圧縮）。なお、ラスタライズされた画像は Compression 設定に従い処理されます。

タイトルの「アプリサイズが 50MB 増えた」原因の本質はここにあります。格納する SVG ファイル自体のサイズが小さくても、ラスタライズ処理などにより、実際のアプリサイズは肥大化します。

4.2 PDF, SVG 形式などのベクター画像と Preserve Vector Data の影響

ベクター画像なら、拡大しても表示品質を保てる。そう考えがちですが、Asset Catalog に格納しただけでは十分ではありません。そこで利用するのが Preserve Vector Data です。Asset Catalog でこのオプションを有効にすると、本来のサイズを超えて表示する際にランタイムで再度ラスタライズされることで、ジャギーやノイズが抑えられます（図 3）。

ただし、このフラグの有無に関わらず**ビルド時にラスタライズされた画像が生成・保持される点は同じ**です（表 3）。ベクター画像そのものはフラグが有効な場合のみ App Thinning 後も保持されます。

以上より、「アイコン等の比較的単純かつ利用箇所が多岐にわたる画像」を扱う際は有効にすることをおすすめします。

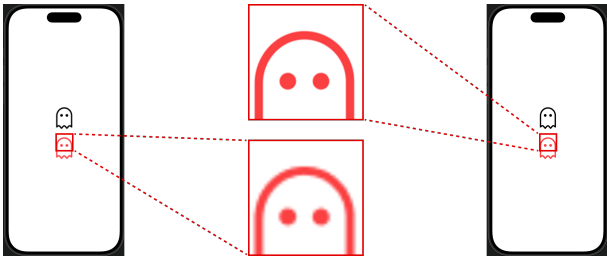


図 3 左：Preserve Vector Data 無効、右：Preserve Vector Data 有効

表 3 Preserve Vector Data のフラグ有無とファイルサイズ（Compression は Inherited 固定）

ファイル形式	Scales	Preserve Vector Data	元素材の合計（MB）	Assets.car サイズ（MB）	Assets.car 増加サイズ（MB）
PDF	Single	True	0.92	4.58	+3.66
PDF	Single	False	0.92	4.58	+3.66
SVG	Single	True	0.38	2.97	+2.59
SVG	Single	False	0.38	2.97	+2.59

^{*4} 全ての検証結果は GitHub(<https://github.com/psbss/iOSDC-2026-Booklet/>) に掲載しています
^{*5} Assets.car に格納された画像データの実体の一つ。スケール・デバイス・GPU 世代などで区別され、それぞれが格納形式や圧縮方法などをもつ

4.3 Single Scale と Individual Scales(1x, 2x, 3x) の使い分け

画像格納時の Scale オプションの選び方は素材の形式と表示対象（場所・端末）によって決まります。

Preserve Vector Data の結果より、ベクター画像を拡大表示する可能性が高い場合は Single Scale で格納します。基本的に同じサイズで表示する場合は、ラスター画像に変換したのち、Individual Scales で格納します。元からラスター画像だった場合も同様です。では、Asset Catalog で Individual Scales として格納する際、全ての倍率（1x, 2x, 3x）を埋める必要があるのでしょうか。

Individual Scales における 1x サイズの画像が利用されるのは macOS（一部の条件下）のみです。iPhone では 2x-3x、iPad では 2x が利用されるため、配信対象が iPad のみの場合は 2x サイズの画像だけでも問題ありません。

では、ラスター画像を Single Scale として格納したらどうでしょうか。この場合は、1x-3x のファイルは生成されず 1 ファイルのみが Assets.car へ格納されます。HEIC のみ例外として、フォールバック用のビットマップ画像が追加で配置されるため Assets.car は素材より大きくなりますが、それでも Individual Scales で格納するよりは小さく、App Thinning を経ることで元ファイルと同等のサイズまで小さくなります。

4.4 App Thinning の影響

ここまで、それぞれのファイル形式、Compression、Scale が Assets.car へどのように影響するか確認しました。ここからは、実際にアプリをインストールするときに必要なサイズを App Thinning の結果から確認します。なお、今回は "iPhone 17 (Device=iPhone18) " 及び "iPad Air 8th Gen (Device=iPad16) " を対象とします（表 4）。

表 4 App Thinning 実行結果 (iPhone 17, iPad Air 8th Gen)

ファイル形式	Scales	格納した画像の合計サイズ (MB)	App Thinning 前 (MB)	iPhone App Thinning 後 (MB)	iPad App Thinning 後 (MB)
JPEG	Individual	6.27	6.70	3.90	2.20
PNG	Individual	4.68	4.20	2.40	1.40
HEIC	Individual	1.07	6.30	0.73	0.52
PDF	Single	0.92	4.90	2.20	1.40
SVG	Single	0.38	3.20	1.70	1.20

App Thinning により、Assets.car には配信先の端末で利用するレンディションのみ含まれるようになったため、全形式でサイズが小さくなりました。この結果は、格納した元ファイル自体のファイルサイズに大きく影響を受けるためです。なお、SVG などのベクターはビルド時にラスターライズされますが、圧縮効率は HEIC 形式に劣るためトータルでは依然として HEIC 形式の方が小さくなります。また、事前に圧縮した場合でも HEIC 形式のサイズがもっとも小さくなる可能性が高いです。これは、画像が持つ色領域にも影響を受けますが、HEIC が非可逆圧縮のファイル形式であり、かつ他形式より数世代新しい高度な圧縮アルゴリズムが採用されているためです。

5 画像格納時の判断基準

以上の検証結果を基にアプリに格納する画像の形式・設定をまとめました（図 4）。特に注意すべきは、素材がベクターだからといって SVG 形式で格納するのは安直だという点です。どの端末に配信するとしても、高解像度な画像は、表示領域が限られている Apple Platform において表示品質の向上に寄与するどころか、ダウンロードサイズを大きくしてしまう可能性があります。

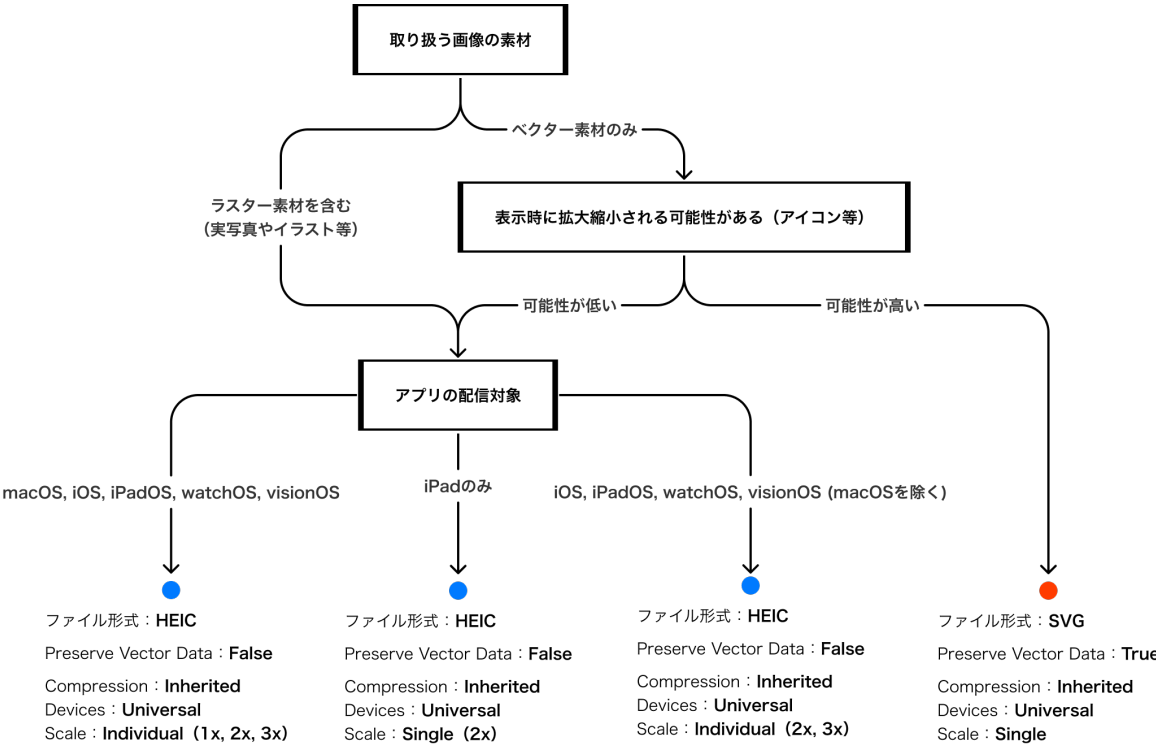


図 4 入稿方法の判断フロー

6 アプリサイズ増加を防ぐ・検知する方法

ここからは、実業務におけるアプリサイズ増加を防いだり、検知したりする方法を紹介します。

6.1 CIでの静的分析

CIでの静的分析には、旧来より用いられてきた Danger^{*6}を使う方法や、今回の調査結果を踏まえたスクリプトを自作する方法があります。スクリプトでチェックする場合は、Asset Catalog 内の Contents.json と画像ファイルを取得して以下を確認します（スクリプトは紙面の都合上省略します）。

- Compression が Inherited かどうか
- SVG：ラスター画像や外部参照を含まない純粋なベクターデータで、Preserve Vector Data が有効かつ Single かどうか
- SVG 以外：ファイルが HEIC 形式で、配信プラットフォームに対応する倍率の画像が登録されているかどうか

6.2 Sentry の Size Analysis を利用して増加を検知する

EmergenceTools（現: Sentry）が提供する Size Analysis を利用することで、継続的な可視化が可能になります（図 5）。XCArchive ファイルをアップロードすることでアプリの推定ダウンロードサイズ、インストールサイズを確認でき、Sentry-CLI や fastlane のプラグインを用いることで簡単に開発フローに組み込み、リリースや PR 単位でトラッキングが可能です^{*7}。

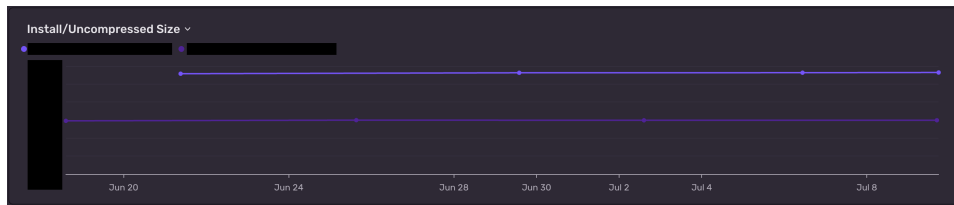


図 5 Sentry Size Analysis によるアプリサイズの可視化 ※ Sentry のプランによって保存期間が異なる

X-Ray という機能を利用すると、サイズの大きいファイルを一覧表示したり、圧縮によって削減できるサイズを可視化したりできます。私たちも現在、アプリサイズ削減の手がかりを得るために利用しています（図 6）。

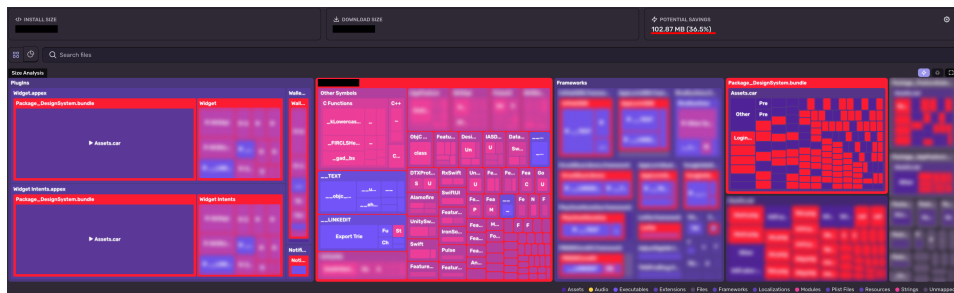


図 6 Sentry Size Analysis X-Ray によるファイル単位の可視化

7 結論

26 年 7 月（Xcode 26.6.0）における画像入稿は、以下の組み合わせが良さそうです。

- 表示時に拡大する可能性があるベクター素材
→ SVG 形式 かつ Single Scale で Preserve Vector Data を有効にして格納する
- 写真やイラストなどのラスター素材や、拡大する可能性が低いベクター素材
→ HEIC 形式 かつ Individual Scales で格納する
- Compression オプションは デフォルト (Inherited) のままにする

8 最後に

画像リソースの追加は、普段何気なく行っている作業の一つですが、思わぬ落とし穴があると判明しました。

日々の積み重ねがダウンロード時間や端末容量に直結します。次に画像を格納する際は、本記事が参考になれば幸いです。

9 参考資料・補足

紙面の都合上、参考資料及び本文中で省略した箇所を GitHub (<https://github.com/psbss/iOSDC-2026-Booklet/>) に掲載しました



^{*6} <https://danger.systems/swift/>

^{*7} <https://docs.sentry.io/platforms/apple/guides/ios/size-analysis/>